

Lab 1: demonstration & defense

Go / Gin / PostgreSQL / Docker / GitHub Actions / Railway

Practical checklist - prepared 3 October 2026

1. Prepare before the lesson

Use PowerShell in the repository directory. Install Go matching go.mod (1.25.6 or newer in the compatible toolchain), Node.js and Docker Desktop. Docker is only needed for the local demonstration. Run the commands in the same terminal so variables remain available.

Option A: deployed API

```
$baseUrl = 'https://lab1-template-production-6bea.up.railway.app'  
curl.exe -i "$baseUrl/api/v1/persons"
```

Expected: HTTP 200 and a JSON array. [] means there are no records; a nonempty list is also valid.

Option B: local Docker Compose

```
docker compose up -d --build  
docker compose ps  
$baseUrl = 'http://localhost:8080'  
curl.exe -i "$baseUrl/api/v1/persons"
```

Expected: app running and postgres healthy. Stop another program using port 8080 before starting Compose. Select one baseUrl and continue with it.

Suggested defense order: 10-15 minutes

1. Architecture and database (2 min).
2. Five CRUD requests and error handling (4 min).
3. Unit tests and Mock Service (2 min).
4. Newman, CI and Railway deployment (3 min).
5. Questions and remaining assignment requirements (2 min).

2. CRUD demonstration: create and read

Create a new demonstration record. Never assume that its ID is 1; read Location from the response.

POST /api/v1/persons

```
$body = @{
    name = 'Ahmed Demo'
    age = 35
    address = 'Moscow'
    work = 'Developer'
} | ConvertTo-Json

$created = Invoke-WebRequest -UseBasicParsing -Method Post `
    -Uri "$baseUrl/api/v1/persons" `
    -ContentType 'application/json' -Body $body

$created.StatusCode
$created.Headers['Location']
$created.Content.Length
$personUrl = $baseUrl + $created.Headers['Location']
```

Expected outputs: 201; /api/v1/persons/<actual ID>; 0. The POST response has an empty body, and Location identifies the created resource.

GET the created person

```
curl.exe -i "$personUrl"
```

Expected: 200, application/json, and the same name, age, address and work as the POST request. The response also contains the generated id.

GET all persons

```
curl.exe -i "$baseUrl/api/v1/persons"
```

Expected: 200 and a JSON array containing the created person. This confirms that data is available through both read endpoints.

What to say

The Handler validates JSON and sends the request to Service. Repository executes a parameterized INSERT in PostgreSQL and returns the generated ID. The Handler builds the Location header.

If Invoke-WebRequest reports an error, stop and inspect its HTTP status and deployment logs before continuing.

3. CRUD demonstration: update, delete, errors

PATCH: update only selected fields

```
$body = @{ name = 'Ahmed Updated'; address = 'Kazan' } |  
    ConvertTo-Json  
$updated = Invoke-WebRequest -UseBasicParsing -Method Patch `  
    -Uri $personUrl -ContentType 'application/json' -Body $body  
$updated.StatusCode  
$updated.Content  
curl.exe -i "$personUrl"
```

Expected: 200. Name becomes Ahmed Updated and address becomes Kazan. Age remains 35 and work remains Developer. The following GET proves that the update was persisted.

Optional request fields use pointers. Omitted age/address/work become nil; SQL COALESCE preserves their existing values. An explicitly supplied age of 0 is still an update. This implementation requires a nonempty name for PATCH.

DELETE and verify absence

```
curl.exe -i -X DELETE "$personUrl"  
curl.exe -i "$personUrl"
```

Expected: DELETE returns 204 with no response body. GET after deletion returns 404. Delete only the demonstration record you created.

Invalid ID and missing name

```
curl.exe -i "$baseUrl/api/v1/persons/abc"  
Invoke-WebRequest -UseBasicParsing -Method Post `  
    -Uri "$baseUrl/api/v1/persons" `  
    -ContentType 'application/json' -Body '{}'
```

Both requests should return 400. Invoke-WebRequest normally throws on an HTTP error; that is expected for this test. curl.exe -i displays error responses without hiding their status.

Status	Meaning
200	Successful GET / PATCH
201	Created; Location header; empty body
204	Deleted; empty body
400	Invalid ID or invalid request JSON
404	Person not found
500	Unexpected service/database error

4. Unit tests and integration tests

Run and count unit tests

```
go test ./...
go test -v ./internal/handler
go test ./internal/handler -list '^Test'
(go test ./internal/handler -list '^Test' |
  Select-String '^Test').Count
```

Expected: PASS / ok, five main test functions and 25 subtests. [no test files] for other packages is normal: the tests live in internal/handler/person_test.go.

Function	Subtests
TestCreatePerson	6
TestGetPersonByID	4
TestGetAllPersons	3
TestUpdatePerson	8
TestDeletePerson	4

Explain the Mock

Unit-test path: HTTP request → Gin router → Handler → Mock Service → recorder. The mock implements the same interface as the real service. `httptest.NewRequest` creates the request; `httptest.NewRecorder` captures the response; `ServeHTTP` runs the Handler without starting a network server.

For POST, explain assertions for 201, the Location header and the empty body. The mock also verifies the input passed to Service. These tests do not verify real SQL or PostgreSQL.

Run Newman against Railway

```
npx.cmd --yes --package=newman@6 newman run `
  "postman/[inst] Lab1.postman_collection.json" `
  -e "postman/[inst][railway] Lab1.postman_environment.json"
```

Expected: five requests, five named assertions, zero failures. Newman exercises the deployed API and real database. The collection creates its own record and deletes it at the end. It does not exhaustively test every possible error case.

5. CI, deployment and troubleshooting

Show GitHub Actions

Open Repository → Actions → CI. For a push or pull request to master, the test job downloads dependencies, runs unit tests, builds the Go application and builds a Docker image.

After Railway reports a successful deployment for upbeat-delight / production, the Newman tests on Railway job runs. It checks API readiness, runs the collection and uploads railway-newman-results (JUnit XML). Actions → CI → Run workflow runs the integration job manually against the current deployment.

Skipped is not failure: the build job is skipped for deployment_status; integration is skipped for push/PR and unrelated deployment events. Look for the successful production Newman run.

Show Railway and the configuration

Show app and Postgres services, the deployment and public domain. The app reads DATABASE_URL and PORT. Inside Compose, the database host is postgres; on Railway the connection comes from the Postgres service reference. Do not reveal passwords.

Name: DATABASE_URL
Value: \${Postgres.DATABASE_URL}

These are two separate variable fields. Do not use Postgres as the variable name, and do not prefix the value with DATABASE_URL=. The app creates persons with CREATE TABLE IF NOT EXISTS before serving requests.

Quick diagnosis

Symptom	Check
502	Deploy logs; database startup; domain Target Port must match PORT.
localhost:5432 refused	DATABASE_URL is missing, empty or still local; apply the variable and redeploy.
Port 8080 already used	Stop the previous local app before Compose.
Classroom check fails	Original classroom.yml uses an empty Heroku baseUrl; separate from successful Railway CI.

Classroom autograding and the Google-sheet marker still need the teacher's configuration. They have not been validated. A green CI run is not proof that all repository checks pass.

6. Defense questions and final checklist

Why use layers?

Handler owns HTTP, Service separates application operations, and Repository owns SQL. The interface allows replacing Service in tests.

Why does localhost fail inside Docker?

It refers to the app container itself. Another container is reached by its service hostname; Railway uses its database connection variable.

Unit tests versus Newman?

Unit tests isolate Handler using mocks. Newman calls the deployed API and verifies interaction with the real database.

What is a multi-stage Docker build?

The Go stage compiles a static binary with CGO disabled. The Alpine stage contains the binary without the Go compiler.

What is CI versus deployment?

CI checks tests and builds. Railway separately builds and runs the app after GitHub updates. A post-deployment event triggers Newman.

Be accurate about assignment compliance

The original assignment specifies Heroku and Docker deployment through GitHub Actions. This implementation uses Railway GitHub autodeploy. Ask the teacher whether that substitution and deployment method are accepted. The original Classroom grading workflow also remains unresolved. Do not claim full compliance before these points are agreed or implemented.

Finish the demonstration

Confirm: CRUD status codes shown; demo person deleted; unit tests passed; Newman has zero failures; CI and post-deployment run explained; remaining requirements disclosed.

```
docker compose down
```

This stops local containers and preserves the database volume. Avoid down -v if you need to keep the data.